Angular Frontend Component Documentation

Modules Used Inside Component Files (.ts)

- Angular Core (@angular/core) – Provides Component decorator and core functionality.
- FormsModule (@angular/forms) – Enables template-driven forms and two-way data binding.
- CommonModule (@angular/common) – Provides structural directives like *ngIf for conditional rendering.

1. Angular Standalone Component Architecture

- Standalone component configuration – Defines the component without requiring an NgModule.
- Component decorator structure – Specifies metadata such as selector, template, and styles.
- Selector usage – Allows the component to be used as a custom HTML tag.
- Inline template and styles – Enables writing HTML and CSS directly inside the component file.
- Module-free setup – Builds components independently without traditional modules.

2. Forms & Validation System

- Template-driven forms – Forms created using directives in the HTML template.
- ngForm directive – Tracks overall form state and validity.
- ngModel two-way binding – Synchronizes data between input fields and component model.
- Form submission (ngSubmit) – Executes a function when the form is submitted.
- Required validation – Ensures mandatory fields are filled.
- Minlength validation – Restricts input to a minimum character length.
- Email validation – Validates proper email format.
- Pattern validation – Uses regular expressions for custom validation rules.
- Error handling using *ngIf – Displays validation messages conditionally.
- Form state tracking – Monitors touched, valid, invalid, and submitted states.

3. Event Handling Mechanism

- Click event binding – Triggers actions when buttons are clicked.
- Submit event handling – Executes logic during form submission.
- Method invocation from template – Calls TypeScript functions directly from HTML.
- User interaction processing – Handles user input actions dynamically.

4. Data Handling (Frontend Only)

- TypeScript object modeling – Organizes data using structured objects.
- In-memory data storage – Stores data temporarily within the application.
- Temporary runtime data – Keeps data only during execution.

- No backend dependency – Operates fully on the frontend.
- Service-based data sharing – Shares data between components using services.

5. Conditional Rendering

- *ngIf directive – Shows or hides elements based on conditions.
- Dynamic UI updates – Automatically updates the view when data changes.
- Error message display logic – Displays messages only when validation fails.
- DOM state control – Controls element visibility using conditions.

6. UI Styling & Responsiveness

- Component-level CSS – Applies styles directly inside the component.
- Flexbox layout – Arranges elements in flexible layouts.
- Hover effects – Changes appearance on mouse interaction.
- Transitions and animations – Adds smooth visual effects.
- Responsive design concepts – Adjusts layout for different screen sizes.

7. Component Metadata Configuration

- Selector property – Defines the custom HTML tag name.
- Imports array – Lists required Angular modules.
- Template configuration – Contains the component's HTML structure.
- Styles configuration – Defines the component's CSS.
- Standalone structure – Makes the component self-contained and independent.